Start here

Every page looks the same as before. Underneath, the code follows one small set of rules, with real data, real security, and no copied code. This file tells you what to read, in what order, and how we work from now on.

What the project has so far

Real (saved in the database, rules and security done)	Still example data (`app/Mock/`) — the tasks left
Landing page + honours board	Achievements pages (B-3)
Sign in, sign up (email code), forgot password	Feedback (B-5) · Complaints (B-6)
My Profile (details, photo, password)	Character certificates (B-4)
People: directory, profile pop-up, Add Person, switch off, a leaving teacher's handover	Profile details nobody stores yet (B-11)
Academic: grades, classes, teachers, curriculum	Most dashboard cards and charts (B-10)
Notice Board (post, pin, approve) · Calendar (events per role)	Not built at all: notifications (B-7), permission cards (B-8), year-end (B-9), settings (B-13)
Term Marks + record book (B-1): marks, Publish Term, averages, positions, awards	
Extracurricular (B-2): programmes, joining with the parent's yes, teams, awards, gallery	
Parent Approvals · Child profile · Audit Log	

Every task left is in `team/CHECKLIST.md` → "Backends left to connect", with what is already decided.

1. Get it running (30 minutes)

The repo has two folders: **`code_lecole/`** (the project — every code path in these documents, like `config/people.php`, is inside it) and **`team/`** (these documents). Every command runs inside `code_lecole/`.

You need: Git, PHP 8.2+, Composer, Docker Desktop, Node (only for `bin/check.sh`), and Claude Code.

```
git clone https://github.com/SDJ-Ro/mock.git
cd mock/code_lecole
cp .env.example .env              # add MAIL_DRIVER=log: emails go to storage/logs/mail.log instead of
being sent
composer install                  # Dompdf, for the certificate PDF
docker compose up -d              # MySQL on port 3308 + phpMyAdmin on http://localhost:8086
php database/update.php           # adds every database update + the demo data
php -d upload_max_filesize=6M -d post_max_size=64M -S localhost:8080 -t public
```

Open http://localhost:8080. Every demo account's password is `Password@123` (the IDs are in `database/seed.sql` and `database/updates/003_people_demo.sql`).

Check it is healthy: `bash bin/check.sh` must end with `0 error(s)`.

2. Read in this order (one evening)

#	Read	Why
1	`code_lecole/CLAUDE.md`	The rules. Your AI reads it too, so it builds the same way
2	`team/HOW-IT-WORKS.md`	One request from the browser to the database, file by file. **Viva sheet.**
3	`team/WHY.md`	Why the code is shaped this way (you will be asked "why")
4	`team/GIT.md`	Branches, commits, pull requests
5	`team/handover/{your name}.md`	Your tasks, in order, and a practice pull request
6	`team/CHECKLIST.md` → "Backends left to connect"	Your tasks, with every decision already made
7	`team/AI-GUIDE.md`	**Using ChatGPT, Gemini, Copilot or any AI other than Claude Code**: which files to give it, in order, and the prompts to paste

Reading on paper or giving it to an AI? The same files are in `team/pdf/`, numbered in this order. Your `team/pdf/AI-PACK-<your name>.pdf` is everything an AI needs about your tasks, in one file.

Then follow one request through the code yourself, with Claude:

```
/explain routes/web.php → NoticeController::save()
```

3. The five places every change goes through

A code-check task like "a student must be under 30 to join" or "add a field" always touches the same places:

#	Place	What lives there
1	`config/white_label/rules.php`, `config/{feature}.php`	The number or the list (`rule('age_limit')`)
2	`config/permissions.php`	Who may do it (`can('feature.action')`)
3	`app/Controllers/{Feature}Controller.php`	Reads the form (`postText()`), checks the rule, refuses with a message or saves
4	`app/Models/{Feature}Model.php`	The SQL, prepared statements, `transaction()`
5	`app/Views/{role}/…` + `app/Views/components/…`	Shows it (`e()`), `$variant` for per-use differences

Plus, for a new column or table: a new file `database/updates/NNN_name.sql`, then `php database/update.php`.

4. How a task goes

1. Pick up your next task from your handover file. Make a branch (`team/GIT.md`).
2. `/next-feature B-5 Feedback` — Claude reads everything and writes one plan. **Don't build yet.** (Another AI: prompt 2 in `team/AI-GUIDE.md`.)
3. Send the plan to the lead (or open a draft pull request with it). Answer the questions in it.
4. Build in small steps; **read every diff before you accept it**; one commit per step.
5. Test: `bash bin/check.sh` (0 errors), every page in the browser, curl attacks (below).
6. Ask Claude's `rules-reviewer` to review the branch. Fix what it finds.
7. Open the pull request, fill in the template, ask the lead to review.

8. After the merge: tick the task in `team/CHECKLIST.md` (in the same pull request).

The curl attacks (every new POST)

```
# 1. no CSRF token → must be refused (example: the teacher's Notice Board save)
curl -i -b cookies.txt -X POST http://localhost:8080/teacher/notice/save -d "title=hi"
# 2. an array where text is expected → must not crash
curl -i -b cookies.txt -X POST http://localhost:8080/teacher/notice/save -d "title[]=x" -d
"_csrf_token=…"
# 3. the wrong role (signed in as a student) → refused
# 4. signed out (no cookie) → refused (a JSON request gets JSON back)
```

Ask Claude to run these for you with a real session cookie: "attack my new routes with curl: no token, field[]=x, wrong role, signed out".

5. Using AI without making a mess

- **Claude Code:** start every session in `code_lecole/`, so Claude reads its `CLAUDE.md` and the project's hooks and skills (it may also read and update `../team/`, e.g. the checklist).
- **Any other AI:** it knows nothing until you give it your `AI-PACK` PDF and the real files — follow `team/AI-GUIDE.md`. It has no edit hook, so run `bash bin/check.sh` yourself after every step.
- **Plan first, build second.** `/next-feature` before any code. Never "just build B-5".
- **Small steps.** One step, read the diff, test, commit. A 2,000-line change can't be reviewed or explained.
- **You must be able to explain every line.** If you can't, ask `/explain` until you can. In the code check you change this code live, without AI.
- **Don't let it touch other people's areas** or "improve" things outside your task. Note them in the checklist instead.
- **Don't accept** inline `style=""`, `$_POST`, SQL in a controller, a typed role list, a typed school name, a number typed into a controller, or a new library. The edit hook and `bin/check.sh` warn you; the lead will refuse it.
- **Never paste** `.env` (passwords, keys) into a chat or a commit.

6. Done means

- [ ] Works for every role that sees it, with real data (the example data in `app/Mock/` deleted)
- [ ] `bash bin/check.sh` = 0 errors (the GitHub check on your pull request is green)
- [ ] curl attacks refused, data unchanged
- [ ] The page looks exactly as before
- [ ] `team/CHECKLIST.md` ticked, new findings added
- [ ] You can explain it and change it live